



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



DYNAMIC 3D EROSION OF TERRAINS

LLUNA CLAVERA COMAS

Thesis supervisor

OSCAR ARGUDO MEDRANO (Department of Computer Science)

Degree

Master's Degree in Innovation and Research in Informatics (Computer Graphics and Virtual Reality)

Master's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Contents

1	Introduction	6
1.1	Contributions	6
1.2	Set-up and tools	7
2	Preliminaries	8
2.1	Terrains and Landscapes	8
2.1.1	General weathering effects	8
2.1.2	Freeze-thaw cycle	8
2.2	Terrain Representation and Modeling	8
2.2.1	Terrain Representations	8
2.2.2	Terrain Synthesis	11
2.3	Model Decompositions	11
2.3.1	Tetrahedralization	11
2.3.2	Voronoi Decomposition	12
3	Previous Work	13
3.1	Work in Applied sciences	13
3.2	Work in Computer Graphics	13
3.2.1	Related Methods in Computer Graphics	13
3.2.2	Weathering in Computer Graphics	14
3.3	Base Simulator for this Project	16
4	Model Decomposition	18
4.1	Initial Method	18
4.2	Problems and Solutions	18
4.3	Final Method	18
4.4	Extensions	18
5	Erosion	19
5.1	Initial Method	19
5.2	Multi-resolution	19
5.3	Physical Model	19
5.4	Extensions	19
6	Implementation	20
7	Results	21



8	Conclusions	22
8.1	Future Work	22
8.2	Conclusions	22

List of Figures

2.1	Heightfield of the mountain Aneto as a monochromatic image and the terrain produced by said heightfield on our application	10
2.2	Illustration of stacked height functions, figure taken from [4]	10
2.3	Example of a tetrahedralization, taken from [9]	12
2.4	Example of a voronoi decomposition of a cube, showing the particles and faces of the decomposition. Figure taken from the documentation of the C++ library Voro++ [3] . .	12
3.1	A welsh dragon breaking. Taken from [12]	14
3.2	Example of wood dedcay of a cut tree trunk. Figure taken from [16]	14
3.3	Arch produced by wind erosion. Figure taken from [17]	15
3.4	Illustration of stacked height functions, used to simulate interactions between vegetation and erosion. figure taken from [6]	16
3.5	Example of terrain enhancenment through adding erosion patterns. Taken from [19] . . .	16
3.6	Example of water infiltrating a terrain divided into a set of voronoi cells connected by links. Taken from [21]	17

List of Tables

Chapter 1

Introduction

Virtual terrains appear in a wide variety of applications: from entertainment to physical simulation or science visualization, a lot of virtual scenes are set on a digital landscape. Despite the ubiquity of virtual terrains on 3D scenes, the complex and fractal structure of natural landscapes still brings multiple challenges for the field of synthetic terrain generation.

Most virtual terrains are represented using 2D heightfields which, despite being highly convenient when generating, storing and manipulating terrain models, have a major drawback: heightfields can only represent 2.5D models. Because each point in a 2D plane has a set elevation and every point between the ground and the given elevation is considered as solid (and the terrain interior), we can not represent features like caves, arches or overhangs, where not all points between the ground and the maximum elevation are solid. Additionally, these representations of terrains also fail to reproduce complex geometry commonly found in rocky terrains, like sharp ridges, loose blocks or steep cliffs.

Many of these complex structures that can not be easily captured by 2D heightfields, arise from different types of weathering events, like percolation and freeze-thaw cycles. To address this limitations on virtual terrain models, our goal in this thesis is to study the mechanical weathering of rocky terrains to enhance models given from 2D heightfields and add the complex features that these fields fail to capture.

Terrain enhancement through weathering simulation not only improves the realism and quality of the models, adding the aforementioned complex features, but it can also be applied dynamically to simulate the changes a terrain goes through over multiple years.

This project is built on a previous project on this same topic [1], using the same algorithm as a base, we worked on addressing some limitations of the previous project.

1.1 Contributions

As mentioned before, this project is heavily based on a previous one, and our proposed algorithm is built on the previous one. Despite that, we add multiple improvements to the base algorithm to increase the quality of the results and its efficiency.

The main point we wanted to address of the previous project was the lack of a physically based approach

to the algorithm, as the proposed algorithm made a lot of simplifications and ignored multiple physical interactions which made the method less accurate and realistic. However, during this thesis we also found other areas that could be improved and expanded to improve both the quality and the efficiency of the algorithm.

The limitations of the previous project and our improvements will be discussed at length during these thesis, but our main contributions can be summarized in three large blocks: improvements to the decomposition of the terrain into cells, improvements in the efficiency of the algorithm and improvement on the quality of the algorithm by extending the simulation to a physically based one.

1.2 Set-up and tools

Another difference between the project we base this thesis on, albeit less significant, is the tools used. In the project that presented the algorithm we are extending, a blender plugging was implemented to show the method, however, here we have implemented our erosion model using C++ and we showed the results in a QT application using OpenGL for rendering.

To make the application, we reused the code provided by an article on terrain descriptors [2], as it already implemented many basic functionalities to work with virtual terrains. We then modified and expanded the application implementing our algorithm for mechanical weathering as well as implementing other functionalities to aid in the visualization and debugging of our method. Some of the changes that we implemented modify the basic functionalities of the program, so our project should not be seen as an extension to said application.

Finally, we used the C++ library Voro++ [3] to generate voronoi decompositions, a core aspect of our erosion algorithm.

Chapter 2

Preliminaries

To understand this project and our contributions to the field, we will first present some background information about the topic, so that the reader can better understand both the problem and our proposed solutions.

2.1 Terrains and Landscapes

T

2.1.1 General weathering effects

2.1.2 Freeze-thaw cycle

2.2 Terrain Representation and Modeling

Virtual terrains are a fundamental aspect of this thesis, so it is important to review current approaches to represent and work with these types of models. A deep dive into the state-of-the-art on terrain generation can be found on [4], but we are going to present only the most relevant methods for our project, as a full review is out of the scope of this thesis.

2.2.1 Terrain Representations

When representing terrains and landscapes there are multiple data structures and representations we can use depending on the ammount of information that we need to store and the nature of the model.

The main model representation techniques can be divided in *Elevation Models*, maninly focused on representing the surface of the terrain, and *Volumetric Models*, that aim to capture some interal properties of the terrain as well as more complex structures.

Elevation Models

Elevation models are the simplest ways of representing virtual terrains. They consist on assigning an height to every point on a 2D space. In a more formal way, a terrain is represented by a function $h : \mathbb{R}^2 \rightarrow \mathbb{R}$ that is at least C^0 . From an height function we can easily extract the slope at one point $s(x, y)$ by using the norm of the gradient:

$$s(x, y) = \|\nabla h(x, y)\| = \left\| \left(\frac{\partial h}{\partial x}, \frac{\partial h}{\partial y} \right) \right\|$$

And from the gradient (projected to $\mathbb{R}^3$) we can easily obtain the normal at one point $n(x, y)$:

$$n(x, y) = \frac{\left(-\frac{\partial h}{\partial x}, -\frac{\partial h}{\partial y}, 1 \right)}{\left\| \left(-\frac{\partial h}{\partial x}, -\frac{\partial h}{\partial y}, 1 \right) \right\|}$$

With the height, normal and slope we have all that we need for a basic representation of a terrain. Of course, as we are using a bivariate function to represent the terrain, we can only have one elevation for each point on the domain, and that makes features like caves or overhangs impossible to represent with this method.

Despite its limitations, elevation models are widely used for its simplicity and memory efficiency as they can capture the general structure of a landscape, given that complex features like overhangs often represent a small part of the overall terrain.

To expand on this topic, elevation models only require a bivariate function to represent the whole terrain, which can capture the whole terrain in a really simple closed-form expression, providing infinite precision with an extremely small ammount of memory. However, constructing functions that represent complex realistic terrains is highly challenging and, for that reason, elevation models commonly use *discrete heightfields* (also called *Digital Elevation Models* or DEM) to represent terrains.

Discrete Heightfields are a collection of altitudes in a 2D grid. In order to create a DEM, you have to divide the domain of the terrain (assuming a rectangular domain) into a grid of $n \times m$ cells, then, you assign an altitude to every cell of the grid. The grid will only contain concrete values for a set of discrete values, but you can obtain a height for every point of the domain by interpolating multiple values of the heightfield. This method limits the accuracy of the model (as we stop having infinite precision), but it has the advantage of representing arbitrary terrains in a really simple way.

Heightfields are an extremely common way to represent terrains, and they can be easily stored in textures and monochromatic images (as figure 2.1 shows), sometimes making use of the texture interpolation capabilities on the GPU to accelerate computations on heightfields. Specifically, DEMs are used on [2], a project focused on terrain analysis that provides multiple metrics for terrains and whose code has been1 reutilized in this project to implement and test our erosion algorithm.

To finalize the topic of elevation models, they have been extended to represent some internal structure of the model by employing a layered representation of the terrain. Layered representations are basically an ordered collection of height functions, each function representing a layer of the model (which usually corresponds to a different material), resulting in a stack of elevations for each grid cell. A representation of these types of layered heightfields can be seen on figure 2.2

Layered representations have been used to simulate erosion ([5] and [6]) and, specially to simulate weathering events related to sediments as those types of rocks are naturally organized in layers. There are interesting works on this topics, but those will be covered on chapter 3

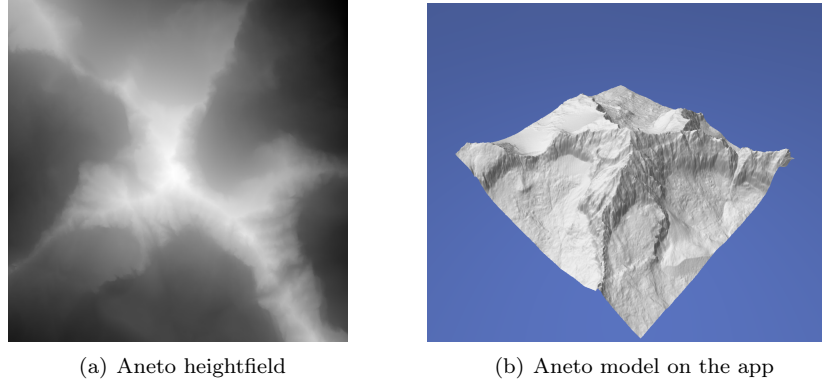


Figure 2.1: Heightfield of the mountain Aneto as a monochromatic image and the terrain produced by said heightfield on our application

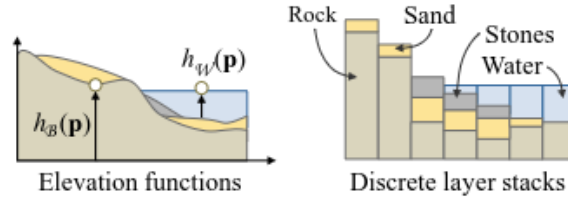


Figure 2.2: Illustration of stacked height functions, figure taken from [4]

Volumetric Models

As seen previously, layered representations are a way to represent different materials and properties of a terrain, but if we are really interested in encoding the internal structure of a landscape, we will have to use volumetric models. Volumetric models can be defined by a function $\mu : \mathbb{R}^3 \rightarrow \mathcal{M}$, where $\mathcal{M}$ denotes the set of materials used (air can count as a material). This function will be assigning every point of the 3D domain to a material, so now complex structures like overhangs can be represented, as well as different internal compositions of the terrain.

Despite volumetric models being defined with a function, in practice we will usually use a 3D grid with discrete samples and perform some interpolation to obtain values outside the fixed grid points, similarly to what we did with DEMs. Each cell of the 3D grid is usually called a *voxel* and the transformation of a model into a set of voxels a *voxelization*. Voxelizations allow us to freely move and remove parts of the model, so they have potential to be used for algorithms that change the structure of the landscape, like for erosion simulation, and they have been explored in the literature [7], but they also have the huge drawback of being extremely expensive memory-wise. For that reason there has been a lot of work in the literature making voxel representations more efficient (like adaptive grids) or presenting novel ways to represent models encoding volumetric information.

For our purposes, we could have used a voxelization to implement our algorithm, but we decided that decomposing the terrain in voronoi cells would produce more natural results in a more efficient manner. The main concepts of the decomposition that we have chosen will be explained on section 2.3.2.

2.2.2 Terrain Synthesis

Terrain generation techniques can be divided in three main groups (although it is common to employ methods from the three groups in the same project):

- **Procedural Generation:** Procedural generation methods artificially generate terrains from scratch by applying randomized processes, like faulting or using noise.
- **Example-based:** Example-based methods create virtual terrains using scanned-data of real-world terrains.
- **Erosion Simulation:** Erosion simulations methods apply algorithms to simulate geomorphological processes when creating the landscape.

Synthetic terrains usually can not represent all the complexity of a natural landscape by procedural generation alone, and that is why erosion simulations are so important, not only in order to "predict" a future for the landscape but also to add realism on virtual terrains. Even example-based terrains often fail to capture some aspects of the terrain they are trying to replicate, as scans have precision limitations (and the bigger the region you want to scan the more difficult it is to do it with accuracy), and erosion simulation can also intervene in those cases to add more complexity to the landscapes.

2.3 Model Decompositions

One of the core aspects of simulating erosion on 3D terrains is the model representation that we use. Any erosion simulation applied on a 3D model will modify the model and its properties, so proper data structures that allow for dynamic modifications of the model are key for these types of simulations. Additionally, applying erosion to an object also exposes the *interior* of said object, so any model representation that we use also has to encode some internal information of the model, as internal points of a model might become part of the surface at some point of the process.

In this section we are going to explain some of the decompositions that can be applied to virtual terrains in order to convert the full model in a set of independent parts that can move freely to modify the overall landscape. We already mentioned voxelizations before (on section 2.2.1), but due to voxelization limitations (mainly the memory cost), we explored more complex techniques to decompose our models.

2.3.1 Tetrahedralization

As you might already know, every polygon can be divided into a set of triangles that exactly match the perimeter of the polygon. Triangulations can also be used to approximate curved surfaces with arbitrary precision. In the case 3D polyhedra and volumes, tetrahedra are the equivalent of triangles, we can approximate any volume using tetrahedra, with more efficiency than with any other polyhedra (like cubes).

We could use a tetrahedralization to decompose our terrain into tetrahedra and, then apply our algorithm over those pieces, and we could even further divide the tetrahedra into more tetrahedra to adjust the granularity of the decomposition. The concept of tetrahedralization and tetrahedral meshes has been used to simulate erosion in the past [8], but we have chosen a different decomposition that has some advantages over tetrahedralizations and produce more irregular shapes, which better represents the shapes we can observe in nature.



Figure 2.3: Example of a tetrahedralization, taken from [9]

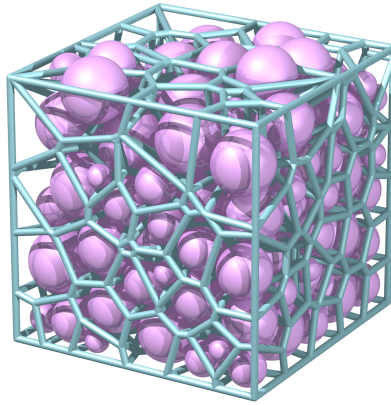


Figure 2.4: Example of a voronoi decomposition of a cube, showing the particles and faces of the decomposition. Figure taken from the documentation of the C++ library Voro++ [3]

2.3.2 Voronoi Decomposition

Given a set of sample points, a voronoi decomposition is a partition of the space into multiple regions, where every region is defined by the set of all points that are closer to a given sample than any other sample. the boundary of every region (or cell), is given by the plane where every point of it is at the same distance of two sample points.

In this thesis we decided to use voronoi cells for multiple reasons. First, they can be generated with a lot of flexibility (just chose an arbitrary set of points) and we can adapt the decomposition depending on our needs. Additionally, the generated cells are not very regular, with varying sizes and shapes, which produces a more natural rocky appearance than other more regular decompositions.

Chapter 3

Previous Work

There has been a multitude of proposed approaches to the problem of erosion simulation, each using different techniques and tackling different parts of the problem. To better understand our method and how it compares to previous solutions, we present a summary of the literature in the field, with their limitations and their differences with our approach. Additionally we will also discuss the paper we are basing our algorithm on [1], as our project can be seen as an extension to their work.

3.1 Work in Applied sciences

Simulation of Crack propagation using eigenerosion

3.2 Work in Computer Graphics

3.2.1 Related Methods in Computer Graphics

In this section we present work that is not focused on terrain erosion but it is still somewhat related to the topic.

Spring-Mass Systems and Brittle Fracture

Solid fracture is a fundamental aspect of simulating rock erosion, specially when simulating the fractures caused by the infiltration of water and its freezing. To simulate breakage one common and simple approach is to use spring-mass systems and remove overly extended springs. This approach was demonstrated years ago in [10] and [11].

A more recent approach in this topic was presented in [12], where they explore the use of peridynamic systems for brittle fracture. Peridynamic systems are spring-mass systems with two particular properties: they decouple stiffness from spring length and they also connect particles just by proximity instead of using 1-ring neighborhoods. An interesting part of the paper is that they have tried using Voronoi-Based Geometry to decompose the model and apply their spring-mass algorithm. In our case we also use voronoi-cells to generate our *masses*, although the dynamics of the links and their fracture are different in our approach. Another approach that uses voronoi geometry for fracture can be found in [13], although in this case they use voronoi polygons over a surface instead of generating voronoi polyhedra in a volume.



Figure 3.1: A welsh dragon breaking. Taken from [12]



Figure 3.2: Example of wood decay of a cut tree trunk. Figure taken from [16]

Modeling Wood Decay

Modeling realistic trees have some parallels to modeling terrains: The goal is to represent a complex natural object that is being constantly modified through the interaction with its environment. Some work has been done [14] [15] on simulating tree decay, which would be the equivalent of simulating erosion on rocky terrains.

On this specific topic, the recent work of [16] (which is yet to be published on SIGGRAPH 2026) presents a really interesting method to simulate wood decay from fungal interaction. The method consists on dividing the tree model into *segments* and then simulating a fungus infiltrating the host and generating fractures in the tree segments. A fungal infiltration also creates conditions facilitating further decay. This is similar to the problem we are tackling: We divide a terrain into different components and then we simulate the water infiltrating the rock, fracturing it and creating conditions where more water infiltration is favoured. The specific mechanics of rock fracture and wood decay are very different, so techniques for wood decay are not really applicable to our work, but the topic has interesting parallels.

3.2.2 Weathering in Computer Graphics

Here we will show a summary of methods for erosion simulation in the literature.



Figure 3.3: Arch produced by wind erosion. Figure taken from [17]

Physics-based Sandstone Simulator

A recent approach to rock erosion was presented by [17]. The goal of the paper was to simulate sandstone dynamics with multiple effects like wind and fluvial erosion. The presented method represents a sandstone structure using a set of particles and then they modify the model using the following equation for each particle:

$$\frac{\partial b}{\partial t} = -\varepsilon(\sigma_c)(\mathcal{W} + \mathcal{F})$$

Where b represents the *viability* which describes the expectancy that the rock has not been eroded, ε represents the *erodibility*, σ_c is the Cauchy stress tensor, and $\mathcal{W}$ and $\mathcal{F}$ represent the wind and fluvial local erosion effects. To compute that equation they divide the process into four parts: First they use the Material Point Method for stress computation and obtain σ_c , then they compute the wind and fluvial erosion components, and finally they perform computations on deposition.

The problem solved by this simulator is similar to the one that we are tackling, but we are dealing with erosion by freezing and not with wind and fluvial erosion so our proposed solution will tackle other situations not covered by this method.

Layered representations to simulate erosion

In section 2.2.1 we briefly discussed layered heightfield representations and how they can be used for erosion simulation, specially to simulate sediments and their interactions. Using this strategy, the work of [6] presents an interesting approach, that specially focus in the interaction between vegetation and erosion and their mutual interaction. In the paper, they divided the landscape in bedrock (unmoving), granular materials that may produce landslides and vegetation (dead and alive) and with that layered representation they could run a simulation to modify those layers, representing the effect of erosion in the ecosystem, improving realism and showing the temporal evolution of a landscape. The layered representation they used can be appreciated on figure 3.4.

In our project we do not intend to deal with sediments and their interaction with the environment, as the main erosion mechanism that we study is the one caused by water entering the pores of the rock and breaking them from inside, causing fractures and potentially separating chunks of rocks from the main structure. Layered representations can represent different features of the landscape with much more memory efficiency than other approaches and can also be used for erosion simulations, but they have limitations that make them not appropriate for certain types of erosion simulation, specially those that consider damage done on all the terrain and not just on the outer layers.

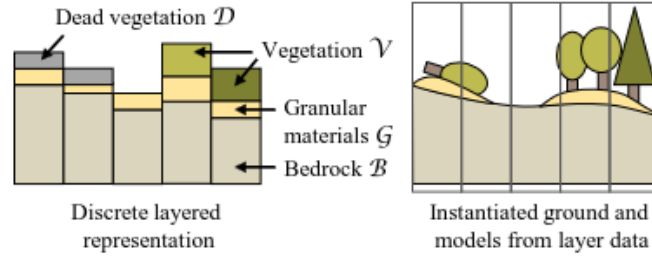


Figure 3.4: Illustration of stacked height functions, used to simulate interactions between vegetation and erosion. figure taken from [6]

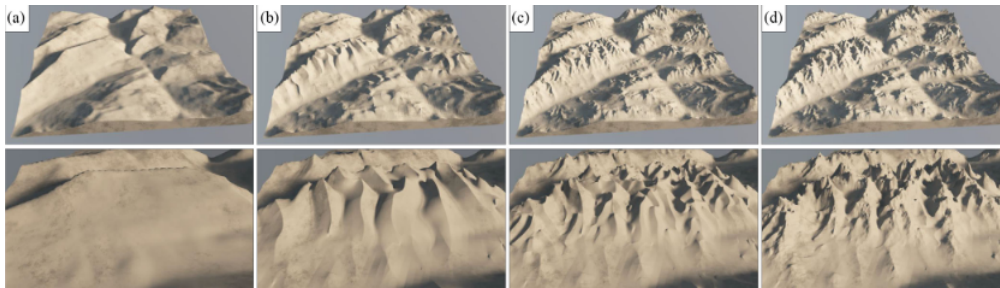


Figure 3.5: Example of terrain enhancement through adding erosion patterns. Taken from [19]

Machine Learning Approach

In the field of terrain erosion there have also been approaches that use machine and deep learning to approximate the effect of erosion on a terrain and improve the realism of the landscape. In our work we do not use any of these processes, so these methods are a little bit far of our proposed solution. Despite that, there are interesting solutions using this approach

In [18] they use a Conditional Generative Adversarial Network trained on real-world terrain to add realistic detail to a rough sketch provided by the user, moreover, they also have a framework to simulate the evolution of the terrain through erosion (with another adversarial network); [19] presents an approach that procedurally enhances terrains by adding erosion patterns using structured noise; and [20] uses Fully Convolutional Neural Networks to upscale low-resolution terrains into plausible high-resolution ones.

3.3 Base Simulator for this Project

As mentioned on the introduction of this report (1), this thesis is built upon the work of [1], so we started from the algorithm they proposed and made some extensions and improvements. For this reason, in order to understand our projet we first have to understand the method presented in the previous project.

In the paper, [1] present an approach to simulate terrain erosion that can be divided into two main parts: first they divide the terrain into independent cells conected by *links* between neighboring pieces, and, then, they apply an erosion algorithm to degrade the links and, eventually, remove the parts that have been disconnected from the terrain. Figure 3.6 shows the erosion simulation in action, with water infiltrating a terrain that has been decomposed into voronoi cells.

To perform the decomposition of the terrain into cells they sampled multiple points of the model and

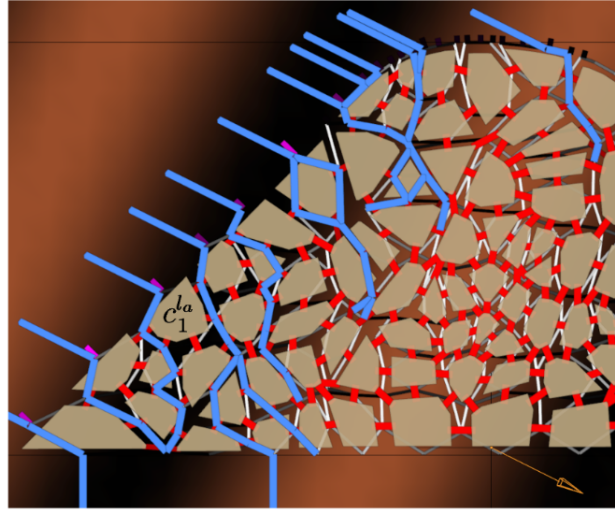


Figure 3.6: Example of water infiltrating a terrain divided into a set of voronoi cells connected by links. Taken from [21]

then generated a voronoi tesellation. In our project we still use the voronoi tesellation to generate the cell graph, but we modified the sampling to obtain better results. The original method used boolean operations between the voronoi tessellation and the original model so that the tesellation would fit the base terrain, but we tried to improve the decomposition so it could better fit the terrain without extra operations.

Regarding the algorithm that simulates water infiltration and link breakage, we implemented a mechanical model to represent link stress and modify those stress values once a link has been broken. This extension of the algorithm now modifies the state of the system after a fracture, instead of just modifying the state of one link, which can produce an effect called crack propagation. The addition of this mechanical model makes the method more based in real natural processes and it improves its realism.

Additionally, we also experimented with multi-resolution decompositions in order to improve the efficiency of the simulation without compromising too much on its quality. The algorithm can be applied in a less granular decomposition and then we can modify the more granular one according to the changes on the higher level.

Furthermore, this thesis also changes a lot the use tools and the testing framework. The original project implemented a blender plugin to demonstrate their solution, while we implemented the algorithm into a C++ OpenGL application.

Finally, even if the basic algorithm is taken from another project, we are going to explain in full detail all parts of the algorithm so that the reader can understand our implementation just by reading this thesis. We are going to highlight our main contributions, but keep in mind some smaller details of the implementation may also be different to the original implementation (and we might not mention the change), as we took the main ideas of the method and implemented them in our own way.

Chapter 4

Model Decomposition

In order to represent the internal structure of a model, we decomposed the base model into multiple cells covering the whole volume of the object. A core part of this project has been focused on the model decomposition, as well as on algorithms to produce and improve said decomposition. In this section of the thesis we are going to introduce some basic topics on 3D modeling, explore the potential representations that could be used for terrain erosion and, finally, we are going to take a deep dive into the method we implemented and the improvements we added to increase the quality of the decomposition.

4.1 Initial Method

4.2 Problems and Solutions

4.3 Final Method

4.4 Extensions

Chapter 5

Erosion

5.1 Initial Method

5.2 Multi-resolution

5.3 Physical Model

5.4 Extensions

Chapter 6

Implementation

Chapter 7

Results

Chapter 8

Conclusions

Summary of the previous results, final conclusions and closing thoughts

8.1 Future Work

8.2 Conclusions

Bibliography

- [1] Diego Mateos et al. “Simulation of Mechanical Weathering for Modeling Rocky Terrains ”. In: *Spanish Computer Graphics Conference (CEIG)*. Ed. by Julio Marco and Gustavo Patow. The Eurographics Association, 2024. ISBN: 978-3-03868-261-5. DOI: 10.2312/ceig.20241139.
- [2] O. Argudo et al. “Terrain descriptors for landscape synthesis, analysis and simulation”. In: *Computer Graphics Forum* 44.2 (2025). DOI: <https://doi.org/10.1111/cgf.70080>.
- [3] ChrisH. Rycroft. *VORO++*. Feb. 2009. DOI: 10.11578/dc.20210416.28. URL: <https://www.osti.gov/biblio/code-54674>.
- [4] Eric Galin et al. “A review of digital terrain modeling”. In: *Computer Graphics Forum*. Vol. 38. 2. Wiley Online Library. 2019, pp. 553–577.
- [5] B. Benes and R. Forsbach. “Layered data representation for visual simulation of terrain erosion”. In: *Proceedings Spring Conference on Computer Graphics*. 2001, pp. 80–86. DOI: 10.1109/SCCG.2001.945341.
- [6] Guillaume Cordonnier et al. “Authoring landscapes by combining ecosystem and terrain erosion simulation”. In: *ACM Trans. Graph.* 36.4 (July 2017). ISSN: 0730-0301. DOI: 10.1145/3072959.3073667. URL: <https://doi.org/10.1145/3072959.3073667>.
- [7] Dennis Wingender and Daniel Balzani. “Simulation of crack propagation through voxel-based, heterogeneous structures based on eigenerosion and finite cells”. In: *Computational Mechanics* 70.2 (2022), pp. 385–406.
- [8] Giuseppe Turini, Fabio Ganovelli, and Claudio Montani. “Simulating Drilling on Tetrahedral Meshes”. In: *Eurographics (Short Presentations)*. 2006, pp. 127–131.
- [9] Matias Muller. *100 X simulation speedup with mesh embedding*. 2022. URL: <https://matthias-research.github.io/pages/tenMinutePhysics/12-softBodySkinning.pdf>.
- [10] Alan Norton et al. “Animation of fracture by physical modeling”. In: *Vis. Comput.* 7.4 (July 1991), 210–219. ISSN: 0178-2789. DOI: 10.1007/BF01900837. URL: <https://doi.org/10.1007/BF01900837>.
- [11] Demetri Terzopoulos and Kurt Fleischer. “Modeling inelastic deformation: viscoelasticity, plasticity, fracture”. In: *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH ’88. New York, NY, USA: Association for Computing Machinery, 1988, 269–278. ISBN: 0897912756. DOI: 10.1145/54852.378522. URL: <https://doi.org/10.1145/54852.378522>.
- [12] Joshua A. Levine et al. “A Peridynamic Perspective on Spring-Mass Fracture”. In: *Eurographics/ACM SIGGRAPH Symposium on Computer Animation*. Ed. by Vladlen Koltun and Eftychios Sifakis. The Eurographics Association, 2014. ISBN: 978-3-905674-61-3. DOI: 10.2312/sca.20141122.

- [13] Saty Raghavachary. “Fracture generation on polygonal meshes using Voronoi polygons”. In: *ACM SIGGRAPH 2002 Conference Abstracts and Applications*. SIGGRAPH '02. San Antonio, Texas: Association for Computing Machinery, 2002, p. 187. ISBN: 1581135254. DOI: 10.1145/1242073.1242200. URL: <https://doi.org/10.1145/1242073.1242200>.
- [14] Adrien Peytavie et al. “DeadWood: Including Disturbance and Decay in the Depiction of Digital Nature”. In: *ACM Trans. Graph.* 43.2 (Feb. 2024). ISSN: 0730-0301. DOI: 10.1145/3641816. URL: <https://doi.org/10.1145/3641816>.
- [15] Bosheng Li et al. “Stressful Tree Modeling: Breaking Branches with Strands”. In: *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*. SIGGRAPH Conference Papers '25. New York, NY, USA: Association for Computing Machinery, 2025. ISBN: 9798400715402. DOI: 10.1145/3721238.3730745. URL: <https://doi.org/10.1145/3721238.3730745>.
- [16] ZHANYU YANG and NIKOLAS A. SCHWARZ. 2026. URL: <https://jango6324.github.io/woodstock/>.
- [17] Zhanyu Yang et al. “Arenite: A Physics-based Sandstone Simulator”. In: *ACM Trans. Graph.* 44.4 (July 2025). ISSN: 0730-0301. DOI: 10.1145/3731201. URL: <https://doi.org/10.1145/3731201>.
- [18] Éric Guérin et al. “Interactive example-based terrain authoring with conditional generative adversarial networks”. In: *ACM Trans. Graph.* 36.6 (Nov. 2017). ISSN: 0730-0301. DOI: 10.1145/3130800.3130804. URL: <https://doi.org/10.1145/3130800.3130804>.
- [19] C. Grenier et al. “Real-time Terrain Enhancement with Controlled Procedural Patterns”. In: *Computer Graphics Forum* 43.1 (2024), e14992. DOI: <https://doi.org/10.1111/cgf.14992>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14992>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14992>.
- [20] O. Argudo, A. Chica, and C. Andujar. “Terrain Super-resolution through Aerial Imagery and Fully Convolutional Networks”. In: *Computer Graphics Forum* 37.2 (2018), pp. 101–110. DOI: <https://doi.org/10.1111/cgf.13345>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.13345>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13345>.
- [21] Diego Mateos Arlanzón. “Simulation of mechanical weathering for modeling rocky terrains”. Projecte Final de Màster Oficial. UPC, Facultat d'Informàtica de Barcelona, Departament de Ciències de la Computació, Oct. 2023. URL: <https://hdl.handle.net/2117/403778>.
- [22] John P. Wilson. “Digital terrain modeling”. In: *Geomorphology* 137.1 (2012). Geospatial Technologies and Geomorphological Mapping Proceedings of the 41st Annual Binghamton Geomorphology Symposium, pp. 107–121. ISSN: 0169-555X. DOI: <https://doi.org/10.1016/j.geomorph.2011.03.012>. URL: <https://www.sciencedirect.com/science/article/pii/S0169555X11001449>.
- [23] Farès Belhadj. “Terrain modeling: a constrained fractal model”. In: *Proceedings of the 5th International Conference on Computer Graphics, Virtual Reality, Visualisation and Interaction in Africa*. AFRIGRAPH '07. Grahamstown, South Africa: Association for Computing Machinery, 2007, 197–204. ISBN: 9781595939067. DOI: 10.1145/1294685.1294717. URL: <https://doi.org/10.1145/1294685.1294717>.
- [24] Mattia Natali et al. “Modeling Terrains and Subsurface Geology.” In: *Eurographics (State of the Art Reports)*. 2013, pp. 155–173.



- [25] Demetri Terzopoulos and Kurt Fleischer. “Modeling inelastic deformation: viscoelasticity, plasticity, fracture”. In: *SIGGRAPH Comput. Graph.* 22.4 (June 1988), 269–278. ISSN: 0097-8930. DOI: 10.1145/378456.378522. URL: <https://doi.org/10.1145/378456.378522>.